```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files

url = "https://raw.githubusercontent.com/justmarkham/DAT8/master/data/Advertising.csv"

try:
    df = pd.read_csv(url)
except Exception:
    print("Upload Advertising.csv")
    uploaded = files.upload()
    df = pd.read_csv(next(iter(uploaded)))

df = df.drop(columns=["Unnamed: 0"], errors="ignore")

print("First five records:")
display(df.head())

print("Last five records:")
display(df.tail())

print("Shape:", df.shape)
print("\nData types:")
print(df.dtypes)

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate records:", df.duplicated().sum())

print("\nSummary statistics:")
display(df.describe())
```

```
Upload Advertising.csv
```
Choose Files advertising.csv

advertising.csv(text/csv) - 4062 bytes, last modified: 10/12/2026 - 100% done

```
Saving advertising.csv to advertising (1).csv
First five records:
```

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9

```
Last five records:
```

	TV	Radio	Newspaper	Sales
195	38.2	3.7	13.8	7.6
196	94.2	4.9	8.1	14.0
197	177.0	9.3	6.4	14.8
198	283.6	42.0	66.2	25.5
199	232.1	8.6	8.7	18.4

```
Shape: (200, 4)

Data types:
TV           float64
Radio        float64
Newspaper    float64
Sales        float64
dtype: object

Missing values:
TV           0
Radio        0
Newspaper    0
Sales        0
dtype: int64

Duplicate records: 0

Summary statistics:
```

	TV	Radio	Newspaper	Sales
count	200.000000	200.000000	200.000000	200.000000
mean	147.042500	23.264000	30.554000	15.130500
std	85.854236	14.846809	21.778621	5.283892
min	0.700000	0.000000	0.300000	1.600000
25%	74.375000	9.975000	12.750000	11.000000
50%	149.750000	22.900000	25.750000	16.000000
75%	218.825000	36.525000	45.100000	19.050000
max	296.400000	49.600000	114.000000	27.000000

```
X = df[["TV", "Radio", "Newspaper"]]
y = df["Sales"]

sns.pairplot(df[["TV", "Radio", "Newspaper", "Sales"]])
plt.show()

plt.figure(figsize=(8, 6))
```

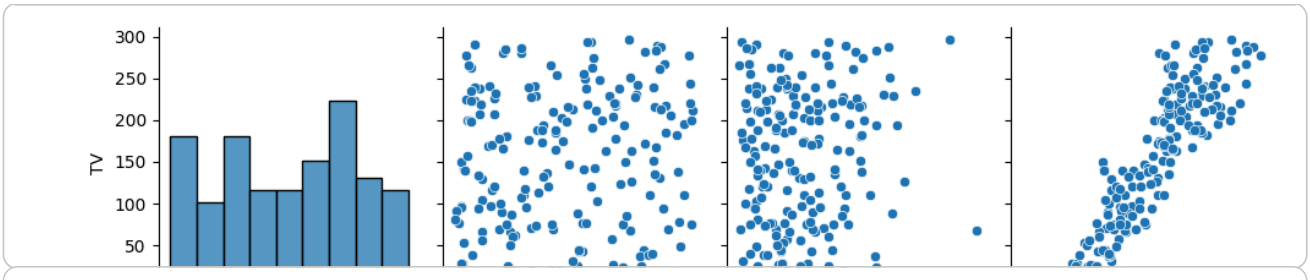
```
sns.heatmap(
    df[["TV", "Radio", "Newspaper", "Sales"]].corr(),
    annot=True,
    cmap="coolwarm"
)

plt.title("Advertising Correlation Heatmap")
plt.show()

correlation = df[["TV", "Radio", "Newspaper"]].corrwith(y)

print("Correlation with Sales:")
print(correlation)

print(
    "Feature most strongly correlated with Sales:",
    correlation.abs().idxmax()
)
```




```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_segment_state=42
)

models = {
    "Linear Regression": LinearRegression(),

    "Polynomial Regression": make_pipeline(
        PolynomialFeatures(degree=2, include_bias=False),
        LinearRegression()
    ),

    "SVR": make_pipeline(
        StandardScaler(),
        SVR(kernel="rbf")
    )
}

predictions = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    predictions[name] = model.predict(X_test)

    print(name, "completed")
```

```
Linear Regression completed
Polynomial Regression completed
SVR completed
```




```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

results = []

for name, pred in predictions.items():
    mse = mean_squared_error(y_test, pred)

    results.append({
        "Model": name,
        "MAE": mean_absolute_error(y_test, pred),
        "MSE": mse,
        "RMSE": np.sqrt(mse),
        "R2 Score": r2_score(y_test, pred)
    })
```

```
results_df = pd.DataFrame(results).set_index("Model")

display(results_df)
```

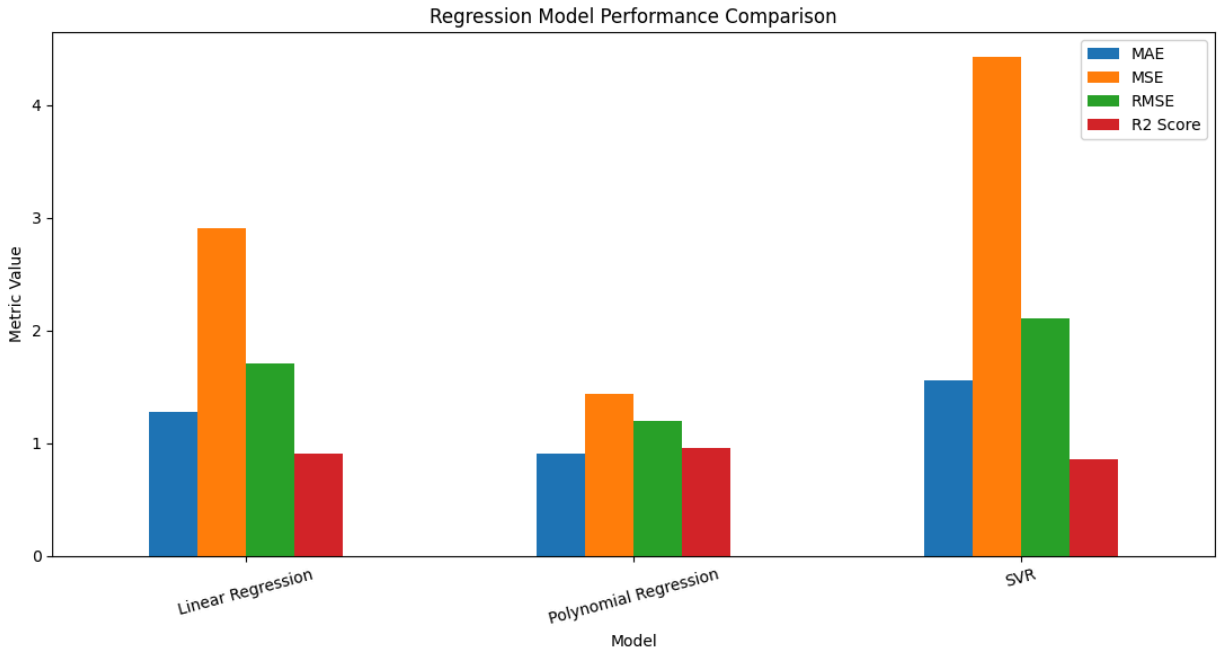
```
dtype: float64
Feature most strongly correlated with target variable: MSE
```

Model	MAE	MSE	RMSE	R2 Score
Linear Regression	1.274826	2.907757	1.705215	0.905901
Polynomial Regression	0.903371	1.442543	1.201059	0.953317
SVR	1.555355	4.421034	2.102626	0.856930

Next steps: Generate code with `results_df` New interactive sheet

```
results_df.plot(
    kind="bar",
    figsize=(11, 6)
)

plt.title("Regression Model Performance Comparison")
plt.ylabel("Metric Value")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```




```
best_model_name = results_df["RMSE"].idxmin()

print("Model with lowest prediction error:", best_model_name)

best_predictions = predictions[best_model_name]

plt.figure(figsize=(7, 6))

plt.scatter(y_test, best_predictions)

minimum = min(y_test.min(), best_predictions.min())
maximum = max(y_test.max(), best_predictions.max())

plt.plot(
    [minimum, maximum],
```